

CRUD de estudiantes

Explicación del index principal del controlador y del modelo

Proyecto MVC_EJEMPLO

Registro, consulta, actualización y eliminación de estudiantes

Reporte de desarrollo y validación de mensajes en la interfaz

10 de septiembre de 2026

Introducción

Este proyecto permite llevar un registro de estudiantes desde una página web. En lugar de guardar la información en hojas separadas, la persona puede consultar una lista, agregar un estudiante, corregir sus datos y eliminar un registro cuando corresponda. Estas cuatro acciones forman el CRUD y son la base del trabajo que se explica en este documento.

Cada estudiante tiene una matrícula, nombre, apellidos, correo, teléfono, carrera y semestre. La matrícula ayuda a distinguirlo y no debe repetirse. Por eso, además de guardar información, el sistema necesita revisar lo que se escribe y avisar cuando algo impide completar una acción.

El desarrollo explica el `index.php` que está fuera de las carpetas, el archivo `EstudiantesController.php` y el archivo `estudiante.php`. La explicación sigue el orden del código y describe en palabras sencillas qué hace cada parte importante. El `index` recibe la solicitud, el controlador revisa los datos y decide qué hacer, y el modelo consulta o cambia la información guardada.

También se comprobó qué ocurre en la pantalla al intentar registrar una matrícula existente, dejar un campo obligatorio vacío, escribir un correo incorrecto y colocar un semestre fuera del límite. Se corrigió la ubicación del mensaje de error para que pueda leerse dentro del formulario. Los avisos utilizados son los normales de la página y del navegador, sin agregar `SweetAlert`.

Índice

Introducción	2
Desarrollo y funcionamiento del proyecto	4
Explicación del index principal	5
Explicación del controlador	6
Registro actualización y eliminación en el controlador	7
Explicación del modelo	8
Cambios de información en el modelo	9
Validaciones en la interfaz	10
Evidencia de la matrícula repetida	11
Conclusión	12
Referencias	13

Desarrollo y funcionamiento del proyecto

El proyecto revisado está en C:\xampp\htdocs\MVC_EJEMPLO. La pantalla se abre desde http://localhost/MVC_EJEMPLO/ y presenta una lista de estudiantes. Desde esa misma pantalla se puede buscar, abrir el formulario de registro y utilizar los botones de edición y eliminación de cada estudiante.

Al seleccionar Nuevo estudiante aparece un formulario vacío. Cuando se presiona Guardar, la página envía los datos al index principal. Este reconoce que se quiere guardar un estudiante y llama al controlador. El controlador revisa la información antes de pedirle al modelo que la guarde. Al terminar, se devuelve una respuesta que permite mostrar si el registro se realizó o si hubo un problema.

Al editar, primero se pide la información del estudiante seleccionado. El formulario se llena con sus datos actuales y conserva su número interno, llamado id. Ese número permite saber exactamente qué registro se debe modificar. No es lo mismo que la matrícula: el id identifica la fila dentro de la información guardada y la matrícula identifica al estudiante dentro del registro escolar.

La búsqueda permite encontrar coincidencias en la matrícula, el nombre, los apellidos o la carrera. Si no se escribe nada, se muestran todos los estudiantes. La lista se ordena por el id del mayor al menor, de modo que los registros con números más recientes aparecen primero.

Para eliminar, la página pide una confirmación antes de enviar la solicitud. Si la persona cancela, no se envía la orden de borrar. Si acepta, se manda el id del estudiante y el modelo recibe la instrucción de eliminar ese registro.

La explicación del código se concentra en los tres archivos indicados. La pantalla se revisa únicamente para describir y comprobar sus mensajes. Las referencias al manual de PHP ayudan a aclarar algunas instrucciones; la descripción del funcionamiento concreto corresponde a los archivos del proyecto MVC_EJEMPLO (2026).

Explicación del index principal

El archivo index.php de la carpeta principal es la entrada del proyecto. La primera línea, `<?php`, indica que comienza el código que se ejecuta antes de responder a la página. La instrucción `declare(strict_types=1)` hace más estricta la revisión del tipo de valores que se entregan a las funciones desde este archivo. Por ejemplo, ayuda a respetar cuándo se espera un número o un texto.

El texto entre `/**` y `*/` es un comentario. Sirve para explicar el propósito del archivo y no ejecuta acciones. Después aparecen tres instrucciones `require_once`. La primera carga Database.php, la segunda carga estudiante.php y la tercera carga EstudiantesController.php. `__DIR__` señala la carpeta donde está este index. `require_once` evita cargar otra vez un archivo que ya se había incluido (The PHP Documentation Group, s. f. c).

La línea que comienza con `$action` guarda la acción solicitada. Primero busca `action` entre los datos enviados por el formulario mediante `$_POST`. Si no está ahí, lo busca en la dirección mediante `$_GET`. Los signos `??` permiten pasar a la siguiente opción cuando falta el valor. Si no se recibió ninguna acción, se utiliza `null`, que aquí significa que no hay una acción indicada.

La condición `if ($action !== null)` revisa si existe una acción. Si existe, `header` indica que se enviará una respuesta de datos en un formato que la página puede leer, llamado JSON, y que el texto admite acentos. Dentro de `try` se intenta abrir la conexión con la información guardada. `$db` conserva esa conexión y `$controller` crea el controlador, entregándole un modelo Estudiante que ya puede usarla.

La parte `match ($action)` elige el trabajo solicitado. `listar` pide la lista; `obtener` pide un estudiante; `guardar` manda los datos nuevos; `actualizar` manda los cambios; `eliminar` manda el número del registro que se quiere borrar. Para obtener y eliminar se utiliza `(int)`, que convierte el `id` a número entero. Si el `id` no llegó, se toma 0. Si la acción no coincide con ninguna opción, `default` devuelve `success` en `false` y el mensaje `Accion no valida`.

`catch (Throwable $e)` recibe un fallo ocurrido dentro de `try`. `http_response_code(500)` indica un problema del servidor y `$respuesta` guarda un mensaje de error. Después, `json_encode` convierte la respuesta al formato que entiende la página; `JSON_UNESCAPED_UNICODE` conserva los caracteres como las letras con acento. `echo` envía la respuesta y `exit` termina el trabajo para no añadir la pantalla completa. Si nunca llegó una acción, el último `require` abre `views/estudiantes/index.php` y muestra la interfaz.

Explicación del controlador

El archivo `controllers/EstudiantesController.php` comienza con `<?php` y después define `class EstudianteController`. Esta parte reúne las acciones relacionadas con los estudiantes. El nombre de la clase está en singular aunque el archivo se llame `EstudiantesController.php`. El `index` utiliza el nombre de la clase al crear el controlador.

La línea `__construct(private Estudiante $model)` recibe y conserva el modelo que se usará para trabajar con los datos. `private` significa que esa referencia se utiliza dentro del controlador. Las llaves vacías indican que no hay otras instrucciones de inicio. Cuando aparece `$this->model`, se está usando el modelo que se recibió en esa línea.

`listar()`: `array` prepara la respuesta con la lista de estudiantes. `array` indica que se devolverá un conjunto de valores. `success` en `true` comunica que esa respuesta se preparó correctamente y `data` contiene el resultado de `obtenerTodos`. La expresión `$_POST['buscar'] ?? ''` toma el texto escrito en el buscador o entrega un texto vacío si no se recibió nada. Así, la misma función sirve para buscar o mostrar la lista completa.

`obtener(int $id)`: `array` recibe el número de un estudiante. `$data` guarda lo que devuelve `obtenerPorId`. La instrucción `return $data ? ... : ...` elige entre dos respuestas. Si se encontró el registro, entrega `success` en `true` junto con sus datos. Si no se encontró, entrega `success` en `false` y el mensaje `Estudiante no encontrado`. Esto evita tratar un registro inexistente como si tuviera información.

`validar(array $d)`: `?string` revisa los datos recibidos. `$d` reúne los valores del formulario y `?string` indica que la revisión puede devolver un texto de error o `null` cuando no encuentra un problema. `$required` contiene matrícula, nombre, apellidos, correo, carrera y semestre. El teléfono no aparece porque es opcional.

`foreach` recorre cada nombre de campo obligatorio. La expresión `$d[$campo] ?? ''` toma su contenido o un texto vacío si no existe. `trim` quita los espacios del principio y del final para hacer la comprobación. Si después de eso no queda texto, `return` termina la revisión con `El campo $campo es obligatorio`. El nombre real del campo ocupa el lugar de `$campo`. Esta revisión no modifica el dato original, solamente comprueba si está vacío.

`filter_var` con `FILTER_VALIDATE_EMAIL` revisa si el correo tiene una forma válida; no comprueba que la cuenta exista. El signo `!` hace que se devuelva `El correo no es válido` cuando la revisión falla (The PHP Documentation Group, s. f. b). Después, dos comparaciones verifican si el semestre convertido a entero es menor que 1 o mayor que 12. Si ocurre cualquiera, se devuelve el aviso del límite. Si todo pasa, `return null` indica que se puede continuar.

Registro actualización y eliminación en el controlador

Guardar un estudiante

`guardar(array $d)`: array recibe los datos del nuevo estudiante. En `if ($error = $this->validar($d))`, primero se realiza la revisión y se guarda su resultado en `$error`. Si hay un mensaje, se devuelve `success` en `false` junto con ese mensaje y se detiene el registro. De esta forma, no se intenta guardar información que ya se sabe incompleta o incorrecta.

Dentro de `try` se llama a `insertar($d)` para pedirle al modelo que guarde al estudiante. Si la operación termina sin lanzar un error, se responde `Estudiante registrado correctamente`. Si ocurre un `PDOException`, que es un error relacionado con el acceso a los datos, `catch` devuelve `No se pudo registrar`. Verifica que la matrícula no esté repetida. La matrícula repetida produce este caso porque la tabla no permite repetir ese valor.

Ese aviso no significa que todos los errores al guardar sean necesariamente por una matrícula repetida. El código actual utiliza el mismo texto para cualquier `PDOException` que llegue a este bloque. En la prueba de matrícula existente sí se confirmó el rechazo del registro y la aparición del aviso correspondiente.

Actualizar un estudiante

`actualizar(array $d)`: array empieza con `empty($d['id'])`. Si el `id` falta o se considera vacío, devuelve `ID no válido`. Después se ejecuta la misma revisión de los campos que se usa al registrar. Reutilizar `validar` permite que el registro y la edición compartan las reglas de correo, semestre y campos obligatorios.

Si los datos pasan la revisión, `$this->model->actualizar($d)` manda los cambios al modelo. Cuando no se produce un error se devuelve `Estudiante actualizado correctamente`. Si ocurre un `PDOException`, se responde `No se pudo actualizar`. Verifica la matrícula. Por ejemplo, la matrícula no puede cambiarse por otra que ya pertenece a un estudiante diferente.

Eliminar un estudiante

`eliminar(int $id)`: array pide al modelo que elimine el registro indicado. Si el modelo devuelve `true`, la respuesta dice `Estudiante eliminado correctamente`. En caso contrario, responde `No se pudo eliminar`. La función no comprueba por separado cuántas filas se borraron; utiliza el resultado de ejecutar la instrucción. Las llaves finales cierran esta función y la clase que contiene todas las acciones.

Explicación del modelo

El archivo `models/estudiante.php` define `class Estudiante`. Su tarea es trabajar directamente con la tabla `estudiantes`. El controlador decide cuándo pedir una acción y el modelo contiene las instrucciones para realizarla sobre los datos guardados. `__construct(private PDO $db)` recibe la conexión y la conserva en `$db` para utilizarla durante las consultas y los cambios.

Consultar la lista y buscar

`obtenerTodos(string $buscar = "")`: `array` recibe el texto de búsqueda. `string` indica que se espera texto y `= ""` permite que se llame sin escribir una búsqueda. La condición `if ($buscar !== "")` comprueba si hay algo escrito. Si lo hay, `$sql` guarda una instrucción que empieza con `SELECT * FROM estudiantes`, que pide todos los campos de los estudiantes que cumplan la condición.

La parte `WHERE` limita los resultados. `matricula LIKE :buscar` busca coincidencias en la matrícula. Las siguientes partes, unidas con `OR`, buscan también en nombre, apellidos y carrera. `OR` permite que baste una coincidencia en cualquiera de esos campos. `ORDER BY id DESC` coloca primero los registros con el `id` más alto.

`$this->db->prepare($sql)` prepara la instrucción y la guarda en `$stmt`. `execute` entrega el texto de búsqueda en el lugar marcado como `:buscar`. Los signos `%` alrededor de `$buscar` permiten encontrar coincidencias aunque el texto tenga otras letras antes o después. Por ejemplo, una parte de un apellido puede encontrar el apellido completo. Preparar la instrucción y entregar sus valores por separado ayuda a tratar lo escrito como datos (The PHP Documentation Group, s. f. a).

Si la búsqueda está vacía, se entra en `else`. `query` ejecuta `SELECT * FROM estudiantes ORDER BY id DESC` sin una condición de búsqueda. Finalmente, `fetchAll` recoge todas las filas encontradas y `return` se las entrega al controlador. Si no hay coincidencias, el resultado es una lista vacía.

Consultar un solo estudiante

`obtenerPorId(int $id): ?array` recibe el número del registro y puede devolver sus datos o `null`. La instrucción `SELECT * FROM estudiantes WHERE id = ?` pide solamente la fila con ese `id`. El signo `?` reserva el lugar donde `execute([$id])` coloca el número recibido. `fetch` recoge una fila y la guarda en `$row`. `return $row ?`: `null` entrega esa fila si existe; si no se encontró, devuelve `null` para que el controlador pueda avisarlo.

Cambios de información en el modelo

Insertar información

insertar(array \$d): bool recibe los datos del estudiante y devuelve un resultado de verdadero o falso. \$sql contiene INSERT INTO estudiantes, que significa agregar una fila. Los campos indicados son matricula, nombre, apellidos, correo, telefono, carrera y semestre. No se incluye id porque la tabla genera ese número automáticamente.

VALUES (?, ?, ?, ?, ?, ?, ?) reserva siete lugares, uno por cada campo. prepare deja lista la instrucción y execute recibe los valores en el mismo orden: matrícula, nombre, apellidos, correo, teléfono, carrera y semestre. Mantener ese orden es importante para no colocar un dato en la columna equivocada. return entrega el resultado de ejecutar la instrucción. Cuando la base de datos rechaza una matrícula repetida, el error regresa al controlador.

Actualizar información

actualizar(array \$d): bool usa UPDATE estudiantes SET para cambiar un registro existente. Después de SET se indica qué valor tendrá cada campo. Cada signo ? se completa con el valor correspondiente que se entrega a execute. Se usan los mismos siete datos del registro y, al final, el id del estudiante que se está editando.

WHERE id=? limita el cambio al estudiante seleccionado. Esa condición es la parte que evita modificar toda la tabla. Por eso el id va al final de la lista de valores: corresponde al último signo ? de la instrucción. El resultado que se devuelve indica si la instrucción se ejecutó; por sí solo no demuestra que se haya cambiado una fila, porque también puede enviarse la misma información que ya estaba guardada.

Eliminar información

eliminar(int \$id): bool prepara DELETE FROM estudiantes WHERE id = ?. DELETE FROM pide borrar una fila de la tabla y WHERE señala cuál. execute([\$id]) coloca el número recibido en el espacio reservado y ejecuta la eliminación. La confirmación que ve la persona ocurre antes, en la pantalla; esta función únicamente recibe la orden ya enviada.

Las llaves al final de cada función delimitan dónde terminan sus instrucciones y la última llave cierra la clase Estudiante. Las líneas vacías solo separan las partes para que resulte más fácil leerlas. Los puntos y comas marcan el final de muchas instrucciones; no representan una acción adicional del CRUD.

Validaciones en la interfaz

Se abrió la página del proyecto y se realizaron pruebas desde el formulario de Nuevo estudiante. Para comprobar la matrícula repetida se utilizó 2026001, que ya aparecía en la lista. Se completaron los otros campos obligatorios con datos de prueba y se presionó Guardar. El sistema rechazó el registro y respondió No se pudo registrar. Verifica que la matrícula no esté repetida.

En la primera revisión, ese aviso se colocaba en la pantalla principal y quedaba detrás del formulario abierto. Se agregó un espacio para mensajes dentro del formulario y se dirigieron ahí los errores de registro o edición. Así, la persona puede leer el aviso y corregir los datos sin cerrar la ventana ni perder lo escrito. También se limpia ese espacio al abrir un registro nuevo o entrar a editar.

Después del ajuste se repitió la prueba con la matrícula 2026001. El mensaje apareció en una franja roja dentro del formulario, por encima de los campos. La imagen de la página siguiente muestra el resultado real. Se mantuvieron los avisos normales que ya utiliza el proyecto y no se agregó SweetAlert.

Para comprobar un campo obligatorio, se borró la matrícula y se volvió a presionar Guardar. El navegador señaló el espacio vacío con el mensaje Completa este campo. Esta revisión ocurre antes de enviar los datos. Además, el controlador conserva su propia revisión para rechazar campos que falten o que contengan solamente espacios.

Para revisar el correo se escribió incorrecto. El navegador impidió continuar e indicó que debía incluirse el signo @. Esta prueba confirma que un correo sin la forma requerida no pasa desde el formulario. El controlador también realiza una revisión del correo por medio de filter_var.

Para revisar el semestre se colocó 13. El navegador mostró que el valor debía ser menor o igual a 12. En el formulario el mínimo es 1 y el máximo es 12; el controlador también revisa ese intervalo. Estos avisos ayudan a corregir los datos antes de intentar guardarlos. Las pruebas descritas comprobaron rechazos de información incorrecta y no requirieron borrar estudiantes existentes.

Evidencia de la matrícula repetida

Figura 1

Aviso visible al intentar registrar una matrícula existente

Nuevo estudiante

No se pudo registrar. Verifica que la matrícula no esté repetida.

Matrícula * Nombre * Apellidos *

2026001 Prueba Validación

Correo * Teléfono

prueba@example.com

Carrera * Semestre *

Sistemas 1

Cancelar Guardar

ID	Matrícula	Nombre	Apellido	Carrera	Semestre	Acciones
11	23240027	S...				
10	2026010	D...				
9	2026009	M...				
8	2026008	L...				
7	2026007	D...				
6	2026006	S...				
5	2026005	Luis Ramírez Torres	luis.ramirez@gmail.com	Ingeniería en Sistemas Computacionales	5	
4	2026004	Ana García Sánchez	ana.garcia@gmail.com	Ingeniería en Sistemas Computacionales	4	
3	2026003	Carlos Martínez Pérez	carlos.martinez@gmail.com	Ingeniería en Tecnologías de la Información	3	
2	2026002	María López Hernández	maria.lopez@gmail.com	Ingeniería en Sistemas Computacionales	2	

Nota. Captura de la prueba realizada en el proyecto MVC_EJEMPLO. Se utilizó la matrícula 2026001 y el aviso aparece dentro del formulario, mientras los datos escritos permanecen disponibles para corregirse.

La franja roja informa por qué no se pudo completar el registro. El formulario permanece abierto y el botón Guardar sigue disponible. La persona puede cambiar la matrícula por una que no esté registrada y volver a intentarlo. Esta evidencia corresponde a un mensaje normal de la interfaz, sin SweetAlert.

Conclusión

El CRUD de estudiantes reúne las acciones necesarias para mantener organizada la información: registrar, consultar, actualizar y eliminar. Al revisar el proyecto se puede seguir el recorrido de una solicitud desde la pantalla hasta los datos guardados y entender por qué cada archivo tiene una responsabilidad distinta.

El index principal recibe la acción y la dirige al lugar adecuado. El controlador revisa los datos y prepara la respuesta que recibirá la página. El modelo contiene las instrucciones que consultan, agregan, cambian o eliminan a los estudiantes. Comprender esa relación facilita ubicar dónde se realiza cada parte del trabajo.

Las validaciones también forman parte del funcionamiento del CRUD. Una matrícula repetida no debe producir otro registro y los datos incompletos o incorrectos deben avisarse de forma clara. Las pruebas mostraron el rechazo de la matrícula existente y los avisos para un campo vacío, un correo sin @ y un semestre superior a 12.

El ajuste en la ubicación del mensaje hizo que el error de registro pudiera leerse dentro del formulario. Con ello, la persona sabe qué debe corregir y conserva lo que ya escribió. El proyecto mantiene sus mensajes normales y cumple esta revisión sin incorporar SweetAlert.

Referencias

Proyecto MVC_EJEMPLO. (2026). *Código fuente del CRUD de estudiantes* [Archivos PHP y JavaScript de proyecto local]. C:\xampp\htdocs\MVC_EJEMPLO.

The PHP Documentation Group. (s. f. a). *Consultas preparadas y procedimientos almacenados*. Manual de PHP. <https://www.php.net/manual/es/pdo.prepared-statements.php>

The PHP Documentation Group. (s. f. b). *filter_var*. Manual de PHP. <https://www.php.net/manual/es/function.filter-var.php>

The PHP Documentation Group. (s. f. c). *require_once*. Manual de PHP. <https://www.php.net/manual/es/function.require-once.php>